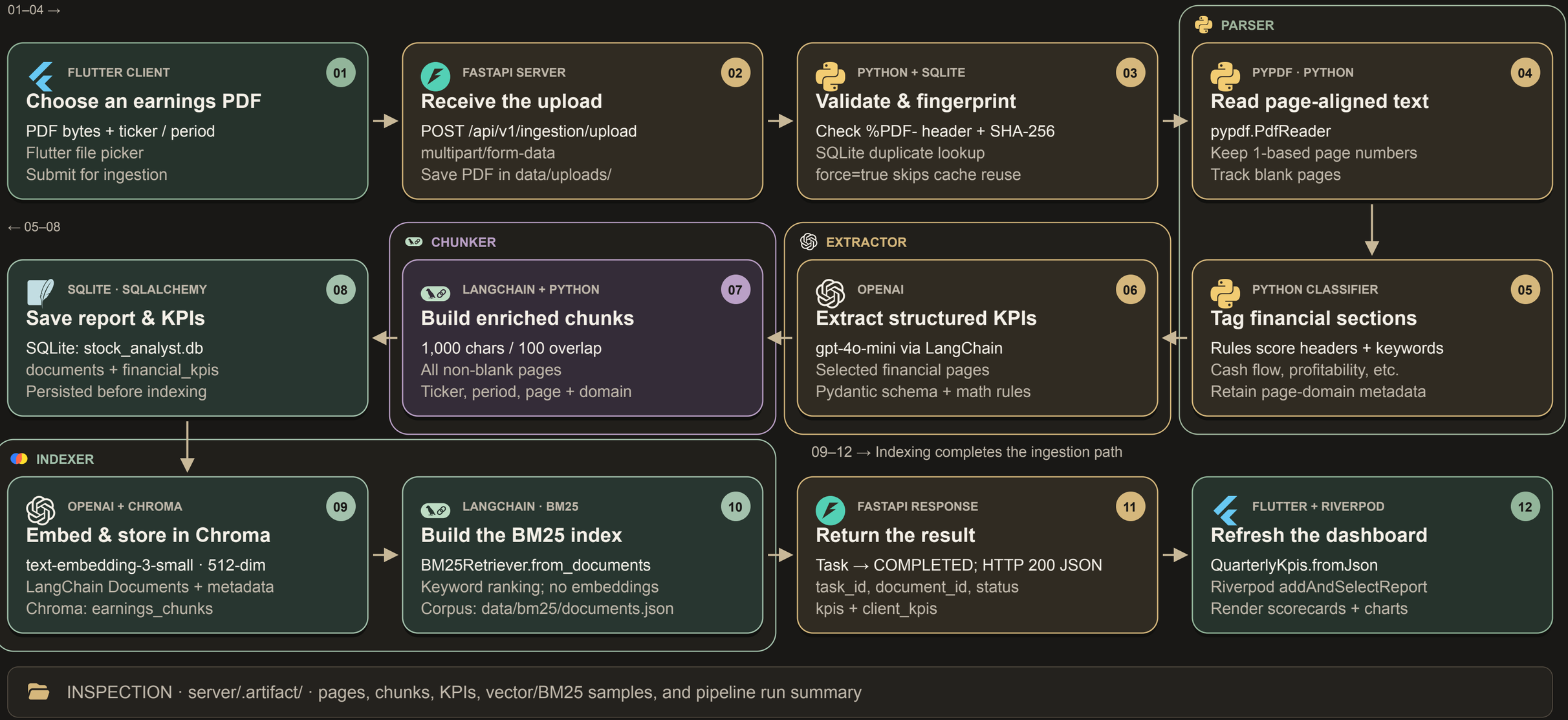
From earnings PDF to searchable evidence

Upload, parse, extract, chunk, persist, and build dense + sparse indexes before returning to Flutter.



What makes ingestion useful for RAG?

Keep small, traceable pieces of evidence that the vector and keyword indexes can search.

PARSER

PYPDF · PYTHON — 01

Keep page boundaries

pypdf extracts page text
Page numbers start at 1
Blank pages are skipped later

PYTHON CLASSIFIER — 02

Keep financial context

Rule-based domain labels
Headers + body keywords
Each page keeps its domain

EXTRACTOR

OPENAI · GPT-4O-MINI — 03

Resolve report identity

KPIs supply canonical metadata
Ticker + fiscal quarter + year
Statement pages sent to LLM

CHUNKER

LANGCHAIN — 04

Split within each page

RecursiveCharacterTextSplitter
1,000 characters per chunk
100-character overlap

PYTHON ENRICHMENT — 05

Attach source metadata

Stable chunk ID + text
Ticker, period, page + domain
Chunk position + char count

Chunking uses all non-blank pages, including narrative pages outside the KPI extraction subset.

A SAVED CHUNK · actual inspection-artifact metadata

```
chunk_id: "TSLA_Q2_2026_p3_c0"
ticker / period: "TSLA" / "Q2" / 2026
page_number: 3
financial_domain: "cash_flow"
chunk position: 0 of 4 · char_count: 943
```

WHERE THE EVIDENCE LIVES TODAY

Chunks feed Chroma + BM25

03b_enriched_chunks_*.json / .md

Written when save_artifacts=true

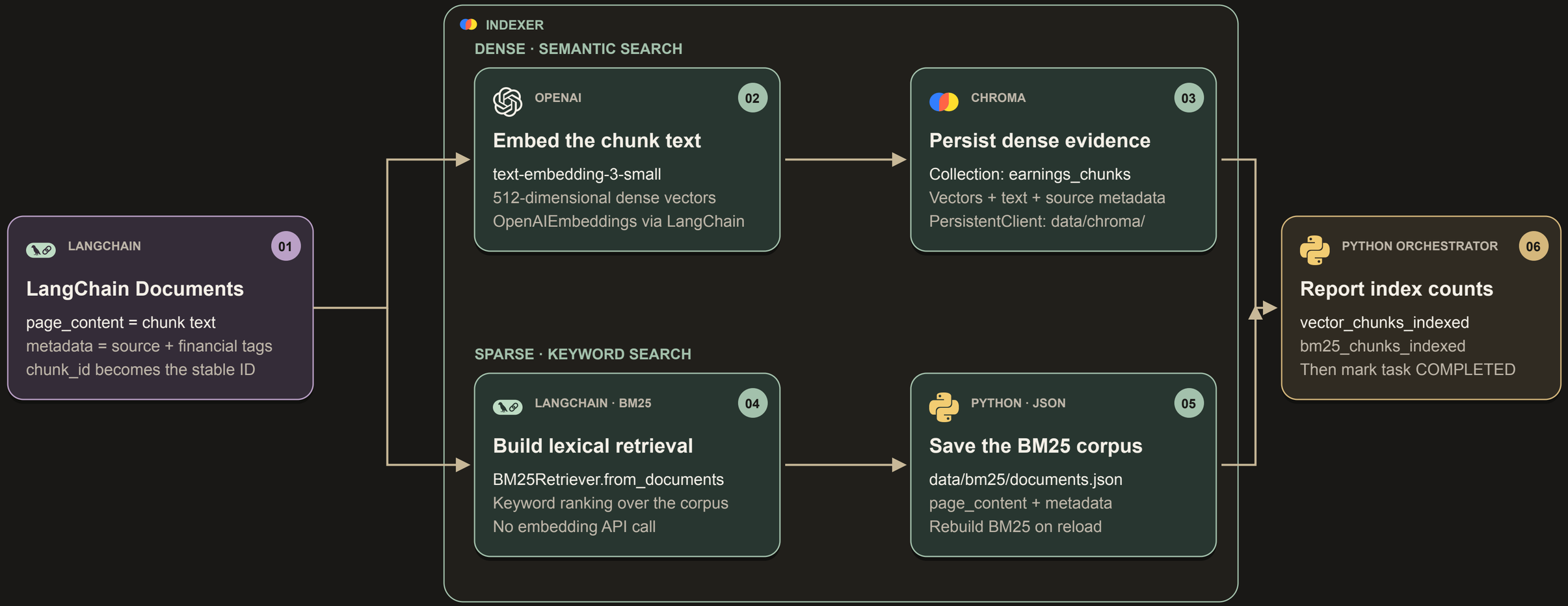
The Flutter upload enables artifact saving.

INDEXING HANDOFF · implemented in the ingestion orchestrator

Same enriched chunks → OpenAI embeddings + Chroma dense index, and LangChain BM25 sparse index

One chunk set, two searchable indexes

Dense vectors capture meaning. Sparse BM25 ranks keyword matches. Both retain source metadata.



The two lanes show shared inputs; the current code attempts vector indexing first, then BM25, sequentially.

Indexing errors are logged and swallowed. COMPLETED alone does not prove both indexes are ready.

SQLite KPIs are already committed before either index is attempted.